COMP2522

Lesson 5: Collections and Iterating

Lesson 5 Topics

- Collections
- Lists, Sets, Maps
- Iterators
- Arrays
- For-Each Loop with Final

Java Collections

1. List
2. Set
3. Map

4. Also, we have the Array type in Java

Array

- Fixed length
- Fixed datatype, including primitives
- Not actually a Java collection, not a class
- Does implement Object methods

Java Array: Simple Array of Strings

```
public class BookStore
{
   private final String[] bookTitles;

   public BookStore()
   {
      bookTitles = new String[2];  // two null references

      bookTitles[0] = "getting real";
      bookTitles[1] = "the $100 startup";
      bookTitles[0] = "the 5AM club";       // overwrites "getting real"
   }
}
```

* bookTitles will <u>always</u> be length 2; it is not possible to add more Strings

"Resized" Array (Not on Quizzes)

```
import java.util.Arrays;

public class BookStore
{
   private final String[] bookTitles = {"getting things done", "the four-hour workweek"};                    // length 2
   private final String[] moreBookTitles;
   private final String[] allBookTitles;

   public BookStore()
   {
      moreBookTitles = new String[2];
      allBookTitles = new String[4];

      moreBookTitles[0] = "getting real";
      moreBookTitles[1] = "the $100 startup";

      allBookTitles = Arrays.copyOf(bookTitles, bookTitles.length + moreBookTitles.length);
      System.arraycopy(moreBookTitles, 0, allBookTitles, bookTitles.length, moreBookTitles.length);
   }

   public static void main(final String[] args)
   {
      final BookStore b;
      b = new BookStore();
   }
}
```

Java Array: while loop

```
int i;
int len;

i = 0;
len = allBookTitles.length;

while(i < len)      // do not call methods in if() statements or loops
{
   final String title;
   title = allBookTitles[i];
   System.out.println(title);
   i++;
}
```

Java Array: while loop

```
int i = 0;

if(allBookTitles != null)
{
   while(i < allBookTitles.length)
   {
      if(allBookTitles[i] != null)
      {
         System.out.println(allBookTitles[i]);
      }
      i++;
   }
}
```

Java Array: for loop

```
if(allBookTitles != null)
{
   for(int i = 0; i < allBookTitles.length; i++)
   {
      if(allBookTitles[i] != null)
      {
         System.out.println(allBookTitles[i]);
      }
   }
}
```

Java Array: for-each loop ("enhanced for")

```
if(allBookTitles != null)
 {
    for(final String title: allBookTitles)    // note the "final"
    {
       if(title != null)
       {
          System.out.println(title.toLowerCase());
       }
    }
 }
```

* This is the best iteration form, better than for loops and while loops

New Example: Array and Foreach Loop

```
class Mathematics
{
   // final means the array can change its contents, but the variable cannot point to another address

   private final double[] mathematicalConstants;

   Mathematics()
   {
      mathematicalConstants = new double[4];

      mathematicalConstants[0] = 3.14159;
      mathematicalConstants[1] = 1.41421;
      mathematicalConstants[2] = 1.61803;
      mathematicalConstants[3] = 2.71828;

      for(double constant: mathematicalConstants)
      {
         System.out.println(constant);
      }
   }

   public static void main(final String[] args)
   {
      final Mathematics math = new Mathematics();
   }
}
```

Java Collections

- Java Collections include three types:
  - **List**
  - **Set**
  - **Map**
  - (Array is NOT a collection type)

  - https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/package-summary.html

- There are different subclasses (subtypes) of each of these, too:
  - ArrayList
  - HashMap

- Java collections can contain only references (not primitives)

Generics: A Short Introduction (more later)

- Generics in Java allow you to create classes, interfaces, and methods that can work with any type of object while ensuring type safety at compile time

- Example:
- Imagine you have a box that can hold one item. Without generics, you could put anything in the box—maybe a toy, a book, or a fruit. But when you take the item out, you don't know what type it is, so you have to guess or check.

- Generics allow you to say, "This box will only hold toys." Now, Java will make sure that only toys are put in the box, and when you take something out, you know it's a toy. This way, there's no need to guess or check—it's safe and predictable.

- Important:
- - Type Safety: Generics ensure that only the type of object you specify can be used. This prevents errors when you try to store or retrieve data.
- - No Type Casting: Without generics, you would need to cast objects when retrieving them (e.g., converting an `Object` to a `String`). With generics, Java already knows the type, so there's no need to cast.

ArrayList Usage

The ArrayList class implements the List interface

1. Import it from the java.util package
2. Declare it as an instance variable (or local variable)
3. Declare its type at the INTERFACE level (i.e. List, not ArrayList)
4. Parameterize its type (e.g. the ArrayList below stores only String references)
5. Instantiate it in the constructor (for example)
6. Use its remove(), add(), get(), size() methods (RAGS); note that there are many more methods too

```
import java.util.ArrayList;

class CarLot
{
   private final List<String> cars;

   CarLot()
   {
      cars = new ArrayList<>();
...
```

ArrayList: ArrayList + Foreach

```
import java.util.List;
import java.util.ArrayList;

public class BookStore{
   private final List<String> titles;

   public BookStore()   {
      titles = new ArrayList<>();

      titles.add("getting things done");
      titles.add("the four-hour workweek");
      titles.add("getting real");
      titles.add("the $100 startup");

      if(titles != null)      {
         for(final String title: titles)         {  // NOTE: final
            if(title != null)            {
               System.out.println(title.toLowerCase());
            }
         }
      }
   }

   public static void main(final String[] args)   {
      final BookStore b = new BookStore();
   }
}
```

*An ArrayList can “always” add more elements into it.

Collections.sort()

- Implement Comparable
- Override compareTo()
- Then use Collections.sort() to sort your collection automatically

- E.g. Student implements Comparable<Student>
- List<Student> students;
- Collections.sort(students);

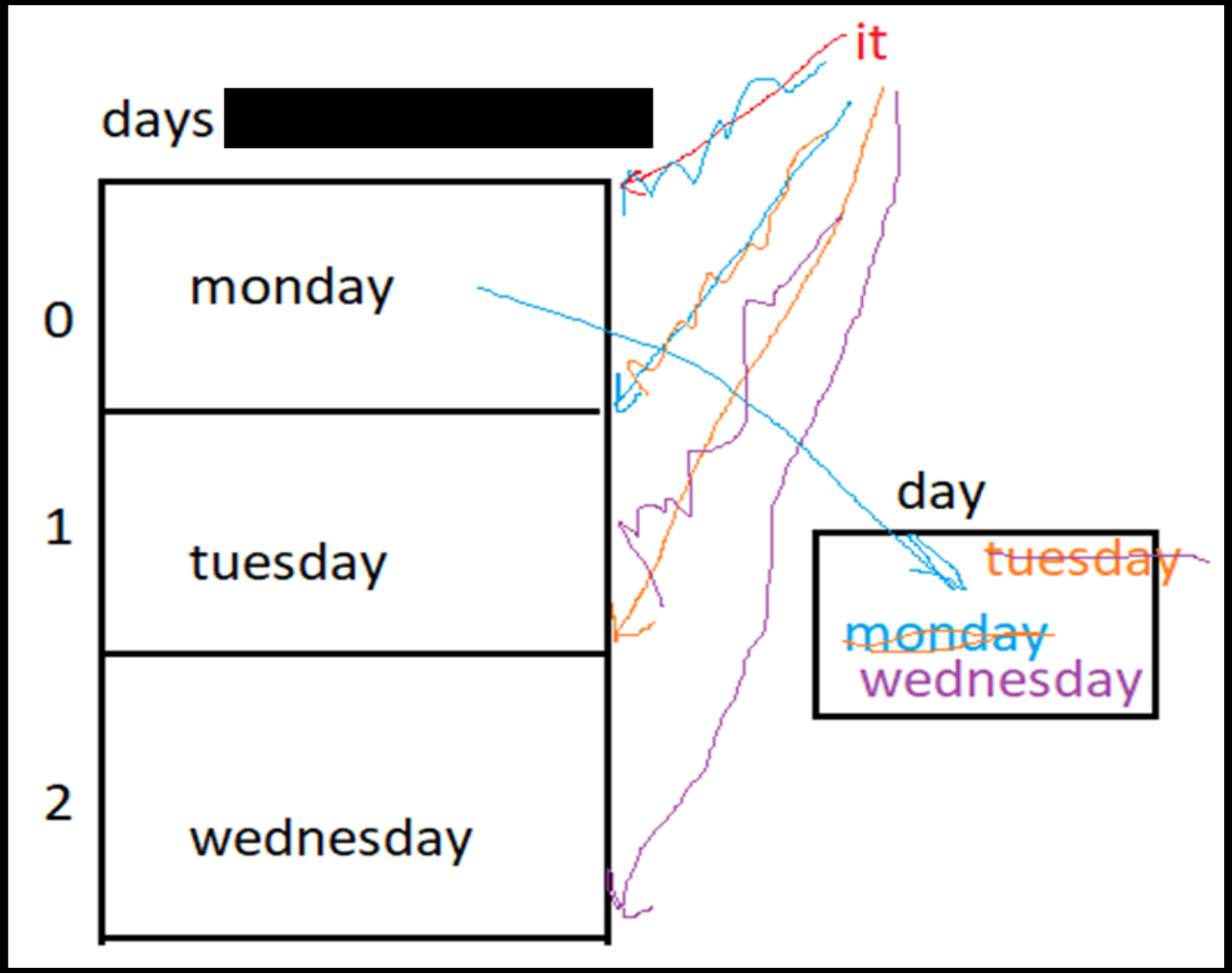
Java Collections: Iterator

Iterator rules:

1. Import from java.util: **import java.util.Iterator;** at top of the class
2. Declare as local variable inside a method
3. Parameterize the iterator (whatever type is in the collection)
4. Combine with while loop: while it.hasNext() next()

5. NOTE: use an iterator only when you want to remove() an element in a loop; a for-each loop crashes if you use the list.remove(): ConcurrentModificationException. Note: it.remove() removes the element you just called next() on.

Iterator


it
days
0
monday
1
tuesday
2
wednesday
day
tuesday
monday
wednesday

Java ArrayList: ArrayList + Iterator

```
Import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;

public class BookStore2
{
   private final List<String> titles;

   public BookStore2()
   {
      titles = new ArrayList<>();

      titles.add("getting things done");
      titles.add("the four-hour workweek");
      titles.add("getting real");
      titles.add("the $100 startup");

      if(titles != null)
      {
         final Iterator<String> it = titles.iterator();

         while(it.hasNext())
         {
            final String title = it.next();  // also advances the iterator
            if(title != null)
            {
               System.out.println(title);
            }
         }
      }
   }
```

New Example: ArrayList + Iterator

Television.java

```
class Television
{
   private String manufacturer;
   private int inches;

   public Television(final String manufacturer,
                     final int inches)
   {
      this.manufacturer = manufacturer;
      this.inches = inches;
   }

   public String getManufacturer()
   {
      return manufacturer;
   }

   public int getScreenSizeInches()
   {
      return inches;
   }
}
```

ElectronicsStore.java

```
import java.util.ArrayList;
import java.util.Iterator;

class ElectronicsStore
{
   private final List<Television> televisions;

   ElectronicsStore()
   {
      televisions = new ArrayList<>();
      televisions.add(new Television("Panasonic", 82));
      televisions.add(new Television("Sony", 55));
      televisions.add(new Television("BENQ", 32));
   }

   public void listAllTelevisions()
   {
      final Iterator<Television> it;
      it = televisions.iterator();

      while(it.hasNext())
      {
         final Television tv;
         tv = it.next();

         System.out.print((tv.getManufacturer() + " "+
                           tv.getScreenSizeInches() + "
                           in")); ....
```

Main.java

```
public class Main
{
   public static void main(final String[] args)
   {
      final ElectronicsStore store;
      store = new ElectronicsStore();

      store.listAllTelevisions();
   }
}
```

Output

```
Panasonic 82 in
Sony 55 in
BENQ 32 in
```

ArrayList RAGS: remove, add, get, size

```
import java.util.ArrayList;

class CarLot
{
   private final List<String> cars;

   CarLot()
   {
      cars = new ArrayList<>();

      cars.add("viper");
      cars.add("accord");
      cars.add("beetle");
      cars.add("charger");

      cars.remove("accord");
      System.out.println(cars.size());

      System.out.println(cars.get(2));
      System.out.println(cars);
   }

   public static void main(final String[] args)
   {
      final CarLot carLot;
      carLot = new CarLot();
   }
}
```

prints:
3
charger
[viper, beetle, charger]

Java Collections and Iterating

- The collections can be iterated similarly to one another, regardless of the underlying collection type
- Sets, Lists, and Maps can **all** use for-each loops and iterators

- List has RAGS (remove, add, get, size) methods
- Map has RPGS (remove, put, get, size) methods (and **keySet**)

- Map keys are unique

Maps and Sets

```
import java.util.HashMap;
import java.util.Set;

class NumberList
{
   private final Map<Integer, String> numbers;

   NumberList()
   {
      numbers = new HashMap<>();
      numbers.put(1, "one");
      numbers.put(2, "two");
      numbers.put(500, "five hundred");
      numbers.put(730, "seven hundred thirty");

      System.out.println(numbers.get(500));
      numbers.remove(500);

      final Set<Integer> allOfTheKeys;
      allOfTheKeys = numbers.keySet();

      for(final Integer key: allOfTheKeys)
      {
         System.out.println(key);
      }

      for(final Integer key: allOfTheKeys)
      {
         System.out.println("key is " + key + " and value is " + numbers.get(key));
      }
   }

   public static void main(final String[] args)
   {
      final NumberList numList;
      numList = new NumberList();
   }
}
```

Maps and Sets

- A map is a collection of key-value pairs
- Only reference types are allowed
- Its keys are unique

- A set is an unordered collection
- No duplicates are allowed

New Example: Map

```
public class Student{

   private final String lastName;
   private final String studentNumber;
   private final int yearOfBirth;
   private double gpa;

   public Student(final String lastName, final String studentNumber,
                  final int yearBorn, final double gradePointAverage)  {
      this.lastName = lastName;
      this.studentNumber = studentNumber;
      this.yearOfBirth = yearBorn;
      this.gpa = gradePointAverage;
   }

   public String getLastName()  {
      return lastName;
   }

   public String getStudentNumber()  {
      return studentNumber;
   }

   public int getYearOfBirth()  {
      return yearOfBirth;
   }

   public double getGpa()  {
      return gpa;
   }

   public void setGpa(double gpa)  {
      this.gpa = gpa;
   }
}
```

```
import java.util.HashMap;
import java.util.Set;

class School
{
   private final Map<String, Student> students;

   School()
   {
      students = new HashMap<>();

      final Student student1 = new Student("woods",
                                    "A00111222", 1975, 3.9);
      final Student student2 = new Student("gates",
                                    "A00333444", 1955, 3.8);

      students.put("A00111222", student1);
      students.put("A00333444", student2);

      students.get("A00333444").setGpa(4.0);

      final Set<String> keys = students.keySet();

      for(final String key: keys)
      {
         final Student student = students.get(key);

         System.out.println("st #" + key + " is " +
               student.getLastName() + " with gpa " +
               student.getGpa());
      }
   }

   public static void main(final String[] args)
   {
      School school = new School();
   }
}
```